

왕복 번역의 기능 보존과 표현 안정화: 세 프로그래밍 언어의 FPS 거리 분석

(Abstract) 코드 왕복 번역에서 기능 보존과 표현의 안정화를 분리하여 측정하였다. C++·Haskell·Prolog 의 공통 알고리즘 문제 15 개를 대상으로 Qwen2.5-Coder 7B 를 사용해 6 방향, 총 90 건의 본 실험을 수행했다. 각 생성 코드에 13 개 평가 입력을 적용하고, 인접 복원 코드의 JPlag text 유사도가 0.85 이상이면 근사 고정점으로 판정했다. 거리는 출발 코드를 포함하여 관측한 서로 다른 언어·소스 상태의 수로 정의했다. 9 건이 기능 검사와 수렴 조건을 함께 통과했으며 성공 거리의 범위는 3~6 였다. 실패의 조기 종료를 짧은 거리로 처리하지 않고, 성공률·성공 조건부 거리·원본 대비 유사도를 함께 보고한다. 결과는 이 모델과 초기 구현에서의 번역 안정성을 나타내며 언어 자체의 보편적 거리로 일반화하지 않는다.

1. 서론

코드 번역은 문법뿐 아니라 자료 구조, 제어 흐름과 입출력 동작의 보존을 요구한다. 자연어 왕복 번역과 고정점 집합(Fixed-Point Set, FPS)의 크기를 이용한 선행 연구[1]를 바탕으로, 실행 가능한 코드의 기능 검사와 근사 수렴을 결합하였다. 번역 실패가 빨리 발생한 경우와 기능을 유지하면서 적은 상태로 안정화한 경우를 구분하는 것이 본 측정의 목적이다.

2. 거리와 실험 방법

출발 언어 A 의 원본을 x_0 , t 번째 중간 언어 B 의 코드를 y_t , A 로 복원한 코드를 x_t 라 두면 경로는 $x_0 \rightarrow y_1 \rightarrow x_1 \rightarrow y_2 \rightarrow x_2 \rightarrow \dots$ 이다. 편도 j 단계에서 생성한 언어·소스를 (L_j, z_j) 라 할 때, 줄바꿈만 LF 로 통일한 소스의 SHA-256 해시 h 로 상태 집합을 정의한다.

$$S_0 = \{(A, h(x_0))\}$$

$$S_j = S_{j-1} \cup \{(L_j, h(z_j))\}, D_j = |S_j|$$

주석·공백·식별자가 다르면 별개 상태이며, 같은 소스도 언어가 다르면 구분한다. 유사도 임계값은 상태 중복 제거에 사용하지 않는다. $A_0 \rightarrow B_0 \rightarrow A_0$ 의 FPS 크기는 2, $A_0 \rightarrow B_0 \rightarrow A_1$ 은 3 이다. 후자에서 $B_0 \rightarrow A_1$ 이 재등장해도 크기는 3 으로 유지된다. 따라서 거리 d_n 은 왕복 횟수나 편도 번역 수와 다르다.

$Sym_adj(t)$ 는 x_{t-1} 과 x_t , $Sym_origin(t)$ 는 x_0 와 x_t 의 유사도다. 모든 생성 단계가 기능 검사를 통과하고, $Sym_adj \geq 0.85$ 및 $D_j \leq 10$ 을 처음 만족한 복원 단계에서 종료하여 $d_n = D_j$, $Sym_final = Sym_origin$ 을 기록한다. 이는 정확한 동일성이나 영구 고정점의 증거가 아닌 운영적 근사 수렴이다. 기능 실패를 먼저 판정하고, FPS=11 이면 해당 코드의 평가 후 추가 번역을 중지한다. 중복·진동은 별도 편도 20 회 예산으로 제한하며 추가 확인 번역은 수행하지 않는다.

P_sf 는 평가 가능한 관측 중 기능과 수렴의 공동 성공 비율이다. 종료까지 모든 생성 코드가 기능 검사를 통과한 비율 $P_functional$ 도 별도로 집계한다. 실패·상한 도달의 성공 조건부 d_n 과 Sym_final 은 NA 로 두고 실제 종료 FPS 크기를 보존한다. 인프라·유사도 오류와 미완료는 평가 실패와 구분한다.

수치·문자열·리스트·동적 계획·탐색·관계의 5 영역에서 번역 전에 각 3 문제를 선정했다. 세 언어 기준 코드 45 개는 동일한 문제당 13 개 평가 입력, 총 585 개 검사를 통과했다. 공개 예제는 프롬프트에 제공하고 평가 입력은 제공하지 않았다. 예비 3 문제×6 방향=18 건은 본 실험과 분리했다. 본 실험은 15 문제×6 방향×1 모델×1 반복=90 건이며, 예비 문제를 제외한 12 문제 분석도 수행했다.

LM Studio 의 Qwen2.5-Coder 7B Instruct(Q4_K_M), temperature=0, top_p=1, seed=20260918, 출력 상한 4,096 토큰을 사용했다. 도구는 GCC 14.2.0, GHC 9.10.3, SWI-Prolog 10.0.2, JPlag 6.3.0 과 OpenJDK 26 이다. 세 언어 모두 text 모드·최소 매칭 9 토큰·averageSimilarity 를 쓰고 입출력 래퍼를 포함했다. 생성 오류는 수정하거나 성공할 때까지 재생성하지 않았다.

3. 실험 결과와 해석

표 1. 방향별 성공률과 성공 조건부 거리·유사도

경로	P_sf [95% CI]	d_n [Q1,Q3]	Sym_final
C→H	4/15 (27%) [10.9,52.0]	4.5 [4,5.25]	0.078
C→P	0/15 (0%) [0.0,20.4]	NA	NA
H→C	5/15 (33%) [15.2,58.3]	5 [4,6]	0.000
H→P	0/15 (0%) [0.0,20.4]	NA	NA
P→C	0/15 (0%) [0.0,20.4]	NA	NA
P→H	0/15 (0%) [0.0,20.4]	NA	NA

C=C++, H=Haskell, P=Prolog. A→B 는 A→B→A 왕복 경로이며 각 15 건이다. 구간은 Wilson 95%, d_n 은 중앙값 [사분위], Sym_final 은 성공 조건부 중앙값이다. NA 는 성공 표본 없음이다.

90 건 중 성공 9 건, 평가 실패 81 건이며 미평가·미완료는 0 건이다. 실패는 컴파일/로딩 오류 28 건, 실행 오류 41 건, 오답 10 건, 시간 초과 2 건이었다. 기능을 통과한 미수렴 관측이 없어 $P_functional$ 은 각 경로의 P_sf 와

같았다. 성공 0 건인 경로의 표본 bootstrap 구간은 [0,0]으로 퇴화하므로 Wilson 구간을 병기했다.

사전 규칙으로 선택한 성공 사례 IPOP_10988(C→H)는 3 회 왕복 후 $d_n=5$, $Sym_adj=1.000$, $Sym_final=0.000$ 였다. 인접 복원 코드의 안정화가 원본 표현의 복원을 뜻하지 않음을 보여준다. 실패의 작은 종료 FPS 는 가까운 언어의 근거가 아니다.

이 펄린드롬 사례의 원본 C++는 양끝 문자를 인덱스로 비교했고, 안정화한 코드는 `std::reverse` 로 만든 문자열과 원문을 비교했다. 두 방식 모두 13 개 입력을 통과했지만 text 의 최소 9 토큰 일치 기준에서 원본 유사도는 0 이었다. 마지막 두 편도의 코드가 이미 관측한 상태로 재등장하여, 6 회 편도 번역과 출발 코드를 포함해도 서로 다른 상태는 5 개였다.

같은 문제의 C→P 경로는 첫 Prolog 코드가 존재하지 않는 `read_line_to_string/1` 을 호출하여 입력 a 에서 실행 오류를 냈다. 관측 FPS 는 2 지만 성공 거리 d_n 은 NA 다. Prolog 를 포함한 네 경로의 성공 0/15 는 이 모델·프롬프트·입출력 계약에서의 관측이며, 모집단 성공률 0 이나 언어 자체의 거리 순위를 뜻하지 않는다.

C→H 와 H→C 의 대응 성공률 차이(C→H 에서 H→C 를 뺀)는 -6.7%p 이고 Bonferroni 구간은 [-33.3, 20.0]%p 였다. 두 방향에서 모두 성공한 문제는 3 개이며 거리의 평균 대응 차이는 0, 구간은 [-2,3]이었다. 이 표본만으로 방향 차이를 확정하기 어렵다.

예비 사용 문제를 제외한 72 건에서는 7 건이 성공했다. 반대 방향 3 쌍은 같은 문제를 대응시켜 비교하고, 성공률은 공통 평가 문제, 거리·유사도는 공통 성공 문제만 사용했다. 문제 단위 bootstrap 2,000 회와 지표별 3 비교 Bonferroni 구간을 적용했다. 공통 성공 표본이 없으면 거리 차이를 산출하지 않았다. 단일 반복으로 실행 간 변동성을 추정하지 않는다.

사전 진단에서 동일 코드의 반복 측정은 1.0 이었지만 곱셈을 덧셈으로 바꾼 코드도 1.0 이었다. Prolog 의 공통 입출력 부분만 남긴 점수도 0.852 였다. JPlag text[2]는 표면 텍스트 척도이므로 이 점수나 임계값을 의미 동등성으로 해석할 수 없다. 기능 검사는 그 한계를 보완하지만 유한한 13 개 입력만으로 모든 입력의 정확성을 보장하지 않는다.

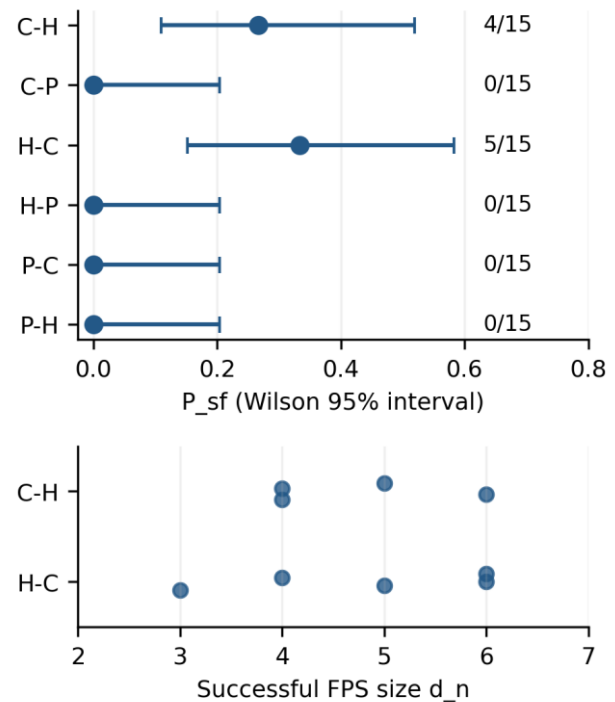


Fig. 1. 방향별 P_{sf} 와 Wilson 95% 구간(위), 성공한 9 건의 FPS 거리(아래). 성공 사례가 없는 경로는 아래 그림에 표시하지 않았다. 점의 작은 세로 이동은 점침을 줄이기 위한 표시이며 값의 변동이 아니다.

4. 결론

RTT의 성공률과 FPS 상태 수를 함께 제시하면 번역 실패와 성공한 표현 안정화를 구분할 수 있다. 다만 d_n 은 성공 사례에 조건부여서 선택 편향을 가지며, 원본 표현과 입출력 래퍼·모델·도구·임계값에 의존한다. 반대 방향은 출발 코드가 달라 대칭성을 전제할 수 없고 삼각부등식도 입증되지 않았다. 따라서 수학적 거리함수나 패러다임의 인과 효과를 주장하지 않는다. 작은 공통 표본과 한 모델의 결과를 다른 모델·실행 반복으로 확장 검증하는 것이 후속 과제다.

참고문헌

- [1] 조찬우 외, "왕복 번역과 고정점을 이용한 언어 거리 계산", 2025 한국소프트웨어종합학술대회 논문집, pp. 1467-1469, 2025.
- [2] JPlag, "JPlag: Source Code Plagiarism Detection", version 6.3.0, <https://github.com/jplag/JPlag> (접근: 2026-09-18).